

Pidiendo Ayuda Con Información Faltante

<https://gemini.google.com/app/4f5aeee7eea6bdb7>

User prompt: como hago esto?

Response: ¡Me encantaría ayudarte! Pero parece que no me llegó la información, el texto o la imagen de lo que necesitas hacer.

¿Podrías darme más detalles o compartir el contexto de lo que quieres resolver?

User prompt: como hago esto indica paso a paso y donde haya que implementar código dime y entrégalo de manera completa

Response: ¡Con gusto te guío paso a paso para resolver la guía!

Dado que el punto 5 menciona una «**especificación adjunta**» (el enunciado concreto del problema o clase Java a implementar) que no aparece en la foto, aquí te entrego la guía paso a paso **completa y estructurada**, con los comandos Git listos y las plantillas para cuando escribas tu código.

Guía Paso a Paso para la Práctica de Git, Java y JUnit

Paso 1: Realizar el *Fork* en GitHub

1. Abre tu navegador e ingresa al repositorio indicado:

```
[https://github.com/GRISE-UPM/2026b-ESPE-parte-c1](https://github.com/GRISE-UPM/2026b-ESPE-parte-c1)
```

2. En la esquina superior derecha de la página, haz clic en el botón **Fork**.
3. Selecciona tu cuenta de GitHub para crear la copia del proyecto en tu perfil.

Paso 2: Clonar el repositorio localmente

Abre tu terminal o consola (Git Bash, Terminal, CMD) y ejecuta:

```
git clone https://github.com/TU_USUARIO_DE_GITHUB/2026b-ESPE-parte-c1.git
cd 2026b-ESPE-parte-c1
```

(Asegúrate de reemplazar `TU_USUARIO_DE_GITHUB` por tu usuario real).

Pasos 3 y 4: Crear README.md y guardar en Git

1. Crea o edita el archivo `README.md` en la raíz de la carpeta escribiendo tu nombre completo:

```
# Nombre y Apellidos
* **Nombre:** Tu Nombre
* **Apellidos:** Tus Apellidos
```

2. Guarda el cambio en Git:

```
git add README.md
git commit -m "docs: añadir README.md con datos del estudiante"
```

Paso 5: Implementar el código (Programación)

1. Lee la especificación del problema dada en tu hoja adjunta.
2. Abre el proyecto en tu IDE (Eclipse, IntelliJ IDEA o VS Code).
3. Implementa el código de producción requerido según la especificación.

Paso 6: Guardar la conversación con el LLM (Programación)

1. Descarga o copia toda la conversación que mantuviste con la IA durante el paso 5.
2. Guarda ese archivo en la raíz del proyecto como `conv_programacion.txt` (o `.pdf`, `.md`, según prefieras).
3. Agrega y realiza el commit del código y de la conversación:

```
git add .
git commit -m "feat: implementar código según especificación y adjuntar chat de programación"
```

Paso 7: Confeccionar las pruebas unitarias con JUnit

1. Crea la clase de prueba dentro del directorio correspondiente (habitualmente en `src/test/java/...`).
2. Utiliza **JUnit 5** (o JUnit 4, según tu entorno) para validar el comportamiento correcto de la clase que creaste en el Paso 5.

Estructura base sugerida para las pruebas:

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class MiClaseTest {

    @Test
    public void testCasosExitosos() {
        // Arrange (Preparación)
        // Act (Ejecución)
        // Assert (Verificación)
        assertTrue(true); // Reemplaza por tu aserción real
    }

    @Test
    public void testCasosLimite() {
        // Prueba de excepciones o valores límite
    }
}
```

Paso 8: Guardar la conversación de pruebas en Git

1. Descarga/copia la conversación que tuviste con el LLM mientras generabas las pruebas.
2. Guarda el archivo como conv_prueba.txt (o .md/.pdf).
3. Confirma los cambios en Git:

```
git add .
git commit -m "test: añadir pruebas unitarias con JUnit y conversación de pruebas"
```

Pasos 9 y 10: Subir los cambios y crear el Pull Request

1. Sube todos tus commits locales a tu repositorio *forkeado* en GitHub:

```
git push origin main
```

(Nota: Si tu rama principal se llama master, usa `git push origin master`).
2. Ve a GitHub a la página de **tu repositorio forkeado**.
3. Verás una notificación o puedes ir a la pestaña **Pull requests** -> clic en **New pull request**.
4. Verifica que el destino (*base repository*) sea `GRISE-UPM/2026b-ESPE-parte-c1`.
5. Haz clic en **Create pull request**.
6. **Importante (Paso 10):** En el **título/comentario del Pull Request**, escribe de forma clara tu **Nombre y Apellidos**.

💡 **Nota:** Si me adjuntas el texto o foto de la especificación técnica del **Paso 5**, puedo escribirte la implementación exacta del código en Java y su respectivo test en JUnit de forma directa.

User prompt: Toma como referencia este documento para el paso 5

Response: ¡Excelente! Con este diagrama UML y las especificaciones detalladas, aquí tienes la implementación Java completa correspondiente al **Paso 5** de la guía (incluyendo la clase principal y las excepciones personalizadas requeridas).

1. Excepciones Personalizadas

Debes crear estas tres clases de excepción (habitualmente en sus propios archivos .java o dentro del mismo paquete):

```
// InvalidContentException.java
public class InvalidContentException extends Exception {
    public InvalidContentException(String message) {
        super(message);
    }
}

// WrongFileTypeException.java
public class WrongFileTypeException extends Exception {
    public WrongFileTypeException(String message) {
        super(message);
    }
}

// WrongEncodingException.java
public class WrongEncodingException extends Exception {
    public WrongEncodingException(String message) {
        super(message);
    }
}
```

```
}  
}
```

2. Tipo Enumerado FileType

```
// FileType.java  
public enum FileType {  
    PROPERTY,  
    IMAGE  
}
```

3. Implementación de la Clase File

Esta es la clase requerida en las especificaciones con todos los métodos e hipótesis de validación del enunciado:

```
// File.java  
import java.nio.file.Path;  
import java.util.ArrayList;  
  
public class File {  
  
    private ArrayList<Character> content;  
    private FileType type;  
    private Path location;  
  
    // Constructor  
    public File(FileType type, Path location) {  
        this.content = new ArrayList<>();  
        this.type = type;  
        this.location = location;  
    }  
  
    // Getters auxiliares  
    public ArrayList<Character> getContent() {  
        return content;  
    }  
  
    public FileType getType() {  
        return type;  
    }  
  
    public Path getLocation() {  
        return location;  
    }  
  
    /**  
     * Añade un par clave=valor al contenido del archivo.  
     */  
    public void addProperty(char[] content) throws InvalidContentException, WrongFileTypeException {  
        // Validación: Content no nulo  
        if (content == null) {  
            throw new InvalidContentException("El contenido no puede ser nulo.");  
        }  
  
        // Validación: Tipo de archivo correcto  
        if (this.type == FileType.IMAGE) {  
            throw new WrongFileTypeException("No se pueden añadir propiedades a un archivo de tipo IMAGE.");  
        }  
  
        // Validación: Formato clave=valor (debe contener un '=')  
        String text = new String(content);  
        if (!text.contains("=") || text.startsWith("=") || text.endsWith("=")) {  
            throw new InvalidContentException("El contenido debe seguir el formato clave=valor.");  
        }  
  
        // Añadir caracteres al contenido existente  
        for (char c : content) {  
            this.content.add(c);  
        }  
    }  
  
    /**  
     * Añade bytes de imagen PNG al contenido del archivo.  
     */  
    public void addImageBytes(char[] content) throws InvalidContentException, WrongFileTypeException, WrongEncodingException {  
        // Validación: Content no nulo  
        if (content == null) {  
            throw new InvalidContentException("El contenido no puede ser nulo.");  
        }  
    }  
}
```

```

    }

    // Validación: Tipo de archivo correcto
    if (this.type == FileType.PROPERTY) {
        throw new WrongFileTypeException("No se pueden añadir bytes de imagen a un archivo de tipo PROPERTY.");
    }

    // Validación: Caracteres en el rango UTF-8 de 8 bits [0, 255]
    for (char c : content) {
        if (c > 255) {
            throw new WrongEncodingException("Caracteres fuera del rango de 8 bits (0-255).");
        }
    }

    // Añadir caracteres al contenido existente
    for (char c : content) {
        this.content.add(c);
    }
}

/**
 * Elimina los últimos 'numberChars' caracteres del contenido.
 */
public void removeContent(int numberChars) {
    if (numberChars <= 0) {
        return;
    }

    // Si numberChars es mayor o igual que el tamaño actual, se vacía completamente el contenido
    if (numberChars >= this.content.size()) {
        this.content.clear();
    } else {
        // Elimina desde el final
        for (int i = 0; i < numberChars; i++) {
            this.content.remove(this.content.size() - 1);
        }
    }
}

/**
 * Métodos stub según el diagrama UML (puedes completarlos si la práctica lo requiere más adelante)
 */
public long getCRC32() {
    return 0L;
}

public long getDiskSize() {
    return this.content.size();
}
}

```

4. Instrucciones para la Entrega en Git (Comandos)

Una vez implementadas las clases en tu proyecto, ejecuta estos comandos en tu terminal para registrar los cambios tal como pide la pauta:

```

# 1. Guardar la conversación de este chat en un archivo
# Guarda este texto/interacción en conv_programacion.txt en la raíz de tu proyecto.

# 2. Añadir y hacer commit en Git
git add .
git commit -m "feat: implementar clase File y excepciones requeridas segun especificacion"

# 3. Avanzar al paso de las pruebas JUnit (Paso 7 del examen)

```

User prompt: a que se refiere con el fichero

Response: Se refiere a los **archivos de texto (.txt) o Markdown (.md)** donde debes guardar una copia completa del chat o conversación que tuviste con la IA (LLM).

En las instrucciones de tu hoja se te piden **dos ficheros** específicos:

1. conv_programacion.<ext> (Paso 6)

- **¿Qué contiene?:** Toda la conversación/chat donde le pediste al LLM que te generara o ayudara a programar el código Java (la clase File, las excepciones, etc.).
- **¿Cómo crearlo?:**

1. Copia todo el texto de esta conversación (o expórtalo si tu interfaz tiene la opción).
2. Crea un archivo en la raíz de la carpeta de tu proyecto llamado `conv_programacion.txt` (o `.md` / `.pdf`).
3. Pega ahí el texto y guárdalo.

2. conv_prueba.<ext> (Paso 8)

- **¿Qué contiene?:** Toda la conversación/chat donde le pidas al LLM que te genere las pruebas unitarias con JUnit para la clase `File`.
- **¿Cómo crearlo?:**
 1. Cuando me pidas (o le pidas a la IA) que genere las pruebas JUnit para la clase `File`, copia esa interacción.
 2. Crea otro archivo en la raíz de tu proyecto llamado `conv_prueba.txt` (o `.md` / `.pdf`).
 3. Pega ahí dicha conversación sobre los tests.

⚠ Muy Importante:

Ambos ficheros deben ser añadidos al control de versiones de Git ejecutando los comandos `git add` y `git commit` antes de realizar el *push* final. De esta forma, el profesor podrá evaluar cómo interactuaste con la IA para resolver el examen.

User prompt: necesito que en base a esto relices el paso 5 y el 7

Response: A continuación se detalla la realización del **Paso 5** y el **Paso 7** según lo indicado en las instrucciones de la práctica:

Paso 5: Diseño y codificación de las pruebas unitarias (JUnit / Mockito)

En este paso se implementan las pruebas unitarias para la clase `File` enfocándose en alcanzar la cobertura de código requerida y la verificación de las excepciones y comportamientos esperados.

A continuación se presenta la implementación para la clase de prueba `FileTest.java`:

```
package es.upm.grise.file;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import es.upm.grise.file.exceptions.InvalidContentException;
import es.upm.grise.file.exceptions.WrongEncodingException;
import es.upm.grise.file.exceptions.WrongFileTypeException;

public class FileTest {

    private FileUtils fileUtilsMock;
    private File file;

    @BeforeEach
    public void setUp() {
        fileUtilsMock = mock(FileUtils.class);
    }

    @Test
    @DisplayName("Debe lanzar WrongFileTypeException si el tipo de archivo no es válido")
    public void testInvalidFileTypeThrowsException() {
        assertThrows(WrongFileTypeException.class, () -> {
            new File(FileType.UNKNOWN, fileUtilsMock);
        });
    }

    @Test
    @DisplayName("Debe crear correctamente el objeto File si el tipo es válido")
    public void testValidFileTypeCreatesInstance() throws WrongFileTypeException {
        file = new File(FileType.TEXT, fileUtilsMock);
        assertNotNull(file);
        assertEquals(FileType.TEXT, file.getType());
    }

    @Test
    @DisplayName("Debe lanzar WrongEncodingException si el encoding del contenido es incorrecto")
    public void testSetContentWrongEncodingThrowsException() throws WrongFileTypeException, WrongEncodingException {
        file = new File(FileType.TEXT, fileUtilsMock);
        String invalidContent = "Contenido invalido";
```

```

        when(fileUtilsMock.checkEncoding(invalidContent)).thenReturn(false);

        assertThrows(WrongEncodingException.class, () -> {
            file.setContent(invalidContent);
        });

        verify(fileUtilsMock, times(1)).checkEncoding(invalidContent);
    }

    @Test
    @DisplayName("Debe establecer el contenido correctamente cuando el encoding es válido")
    public void testSetContentSuccess() throws Exception {
        file = new File(FileType.TEXT, fileUtilsMock);
        String validContent = "Contenido valido";

        when(fileUtilsMock.checkEncoding(validContent)).thenReturn(true);

        file.setContent(validContent);
        assertEquals(validContent, file.getContent());
        verify(fileUtilsMock, times(1)).checkEncoding(validContent);
    }

    @Test
    @DisplayName("Debe lanzar InvalidContentException al procesar si el contenido es nulo o vacío")
    public void testProcessContentWithEmptyContentThrowsException() throws WrongFileTypeException {
        file = new File(FileType.TEXT, fileUtilsMock);

        assertThrows(InvalidContentException.class, () -> {
            file.processContent();
        });
    }
}

```

Paso 7: Ejecución de pruebas y reporte de cobertura

Para ejecutar el conjunto de pruebas unitarias y generar el informe de cobertura, se deben ejecutar los siguientes comandos Maven desde la consola en la raíz del proyecto:

1. Comando de ejecución de pruebas:

```
mvn clean test
```

2. Comprobación y resultados esperados:

- **Pruebas unitarias:** Todos los test definidos en `FileTest` deben ejecutarse y finalizar en estado **BUILD SUCCESS** (0 fallos y 0 errores).
 - **Cobertura de código (JaCoCo / Cobertura):**
 - Se verifica que las rutas principales y de excepción de la clase `File` hayan sido cubiertas mediante el uso de dobles de prueba (Mocks).
 - El informe generado (usualmente disponible en `target/site/jacoco/index.html` tras la fase `test` o `site`) muestra la cobertura de líneas y ramas del código probado.
-